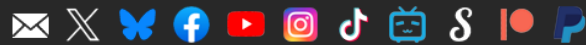


Inigo Quilez :: articles :: compiling small - 2002



As demosceners know, creating a 64kb demo is a triky thing, that requieres some special settings for compiling the code, but most importantly clever designing of the code. Some people think a 64 kb is a small project. Indeed, it's not a big project, but it's not necessarily just 50 files code. Good design requires good modularity. Often even these little demos used workspaces a dozen projects. This can mean, for example, 600 source code files made of 724 thousand lines (5% of comments). Obviously, not all that code is finally going into the final executable. For example, **195/95/256** was compiled from 137 source files, 30 thousand code lines, 120 thousand data lines, 2% comments. The depth of the file tree structure was about 3 to 4 (only the first level is opened in the picture below).

For creating the demo I normally use Visual Studio C++ (Visual Studio Express 2008 today, and Visual C++ 6.0 in the past), and sometimes also an assembler like Nam or Masm for few files of assembly source code. For Linux, Irix and MacOS pure C is used, so everything can be compiled with gcc, Intel Compiler, MipsPro compiler or any other thing able to compile ANSI C code.

All the code of the intro is OS independant, except for the basic functions for opening a window or playing a sound. That code is located in the "sys/" directory, where subdirectories hold the code that implements those functionalities for each OS. That means, that only the "sys/" folder has OS dependant stuff, and the rest of the code can be safely recompiled with 0 warnings in any platform, without need of ugly **#ifdefs** in the middle of the rendering or music playing code, for example. In the "sys/" folder there is also the header file defining the correct data types so that the code works both in 32 and 64 bit compilers/machines.

However, all the tricky compiler options used to make the demo be smaller than 64kb are only done to the Windows version. For other OSs, the demo usually takes arround 75 kb. This is mainly because of two reasons. First, the file compressor we use only works on x86/Windows combination (kkrunchy by ryg/fr), and secondly, we only master the Visual C++ compiler and the way code is generated for Windows platform.

Before going into the details for the VC++ 6.0 compilation, you might wonder why we used this compiler and not a newer version of the VC++ family. Well, experimentation with newer compiler didn't succed on creating smaller executable file than with the 6.0 version. We also used Intel Compiler 7.0, that creates faster executable (specially for the soft synthesis), but the resulting executable was 4 to 5 kilobytes bigger.

So, let's see the details. First, we created three project configurations. The **Release**, the **Debug** and the **ReleaseDebug**. Except for the files in the "sys/" directory, the three configurations compile the same source files. The **Release** configuration contains the settings for the correct creation of small files, and doesn't link to any external library - that's the one we use for the final executable. The **Debug** one creates a standard Windows executable, with all the debugging symbols and information, that helps during the creation process because allows to set breakpoints, resume execution and so on. It also creates a log file, that helps checking for correct execution. This configuration, in the other hand, it also uses file caching of all the generated 3D and 2D content, so that the demo can instantly be initialized during debugging phase. The third configuration, **ReleaseDebug**, is a combination of the other two, where no external libraries are used, all the optimizations are "on", but file caching is still used to speed up loading times.

So that the **Release** creates the smallest possible executable, this compiling options were used:

<u>Purpose</u>	<u>Flag</u>
avoid _ftol()	/Qlfist
fastcall convention	/Gr
no exception Handling	/GX-
basics intrinc	/Oi
disable stack checks	/Gs

Plus the obvious options for:

space optimization	/Os
speed optimization	/Oa /Og

Many people doesn't use the first optimization, /Qlfist, mainly because it's not documented very well. If not used, the C compiler will insert a function call to **_ftol()** each time the C code makes a conversion from floating point to integer. That function is responsible for ensuring the correct conversion as defined by the IEEE floating point standard. That is not only a slow process, but also makes your code dependant on the libc. So many people creates his own **_ftol()** function. Actually, Paradise was using that approach. But later we discovered that with this compiling option, the function call is not generated at all. Thus, code is saved, and more importantly, speed is gained. This is a very nice trick, specially for those 4k intros made in C/C++.

Second non obvious optimization, is to change the default call convention (`__cdecl`) so that parameters are not passed to the functions thru the stack, but thru registers when possible. This, again, makes the code faster, and more importantly, smaller. In 195/95/256, we saved up to 2 kilobytes with this trick.

Third optimization. If a program is well design, there is no need exception handling. Yes. Exception handling is just a workaround for people that incorrectly uses C++ and allocates resources in class constructors (allocations should be done in `create()` methods, and leave the constructors for their original purpose - initialize class members). So, we discard the exception handling, and let's use correctly defined interfaces to track the possible execution errors.

The fourth option, use of `__builtin_` intrinsics, allows the compiler to insert a cpu instruction in the code when possible. If not used, a call to **`sqrtf()`** in the C code will generate a call, while with this option the assembly instruction is generated instead. This way, we save writing ourselves (in assembler) all these functions: **`sinf()`**, **`cosf()`**, **`fabsf()`**, **`memset()`**, **`memcpy()`** and many others. Still, a few functions -as **`fmodf()`**- will have to be done by hand.

Again, if the code is deterministic and well design, no stack checks are needed. So use the `/Gs` option to skip unnecessary code.

The rest are normal compiler optimizations. But, there is something you can still do in the C code to reduce the code size: use always the **`f`** subscript for the floating point constants, otherwise a double will be stored in the executable, and a innecessary conversion will occur. As the size of a double is two times the size of a float, lot of space can be wasted if not carrefully adding the **`f`** to all the constants. Normally, if you follow the rule, you should be able to parse all the assembly listing code and find zero **`QWORDS`** in the code.

The remaining tricks to keep the executable small are just related to the way programming is done, algorithms are design, code is reutilized and data-layout. But please, never discard checking for errors as a way to reduce the size of your code, at least not in a 64 kb unless you are really in the limit (say, 200 bytes away from the 64kb limit, and dead-line in ten minutes).

inigo quilez - learning computer graphics since 1994